

10.1 SSO Integration (Java – JSP & Servlet)

Table of contents

10.1 SSO Integration Example (Java – JSP & Servlet)	1
10.1.1 Prerequisites	2
Example Configuration:	2
10.1.2 Initiate SSO Login (Redirect User to SSO)	4
JSP Login Button	4
Servlet Implementation	4
Explanation	4
10.1.3 Receive Authorization Code from SSO	5
Extract Parameters	5
Validate State (Security Check)	5
Explanation	5
10.1.4 Exchange Authorization Code for Tokens	6
Token Request Code	6
Explanation	6
10.1.5 Receive Token Response	7
Example Response	7
Read Response	7
10.1.6 Decode and Validate ID Token	8
Decode JWT	8
Explanation	8
10.1.7 Token Validation	9
Public Key Endpoint	9
Maven Dependency	9
How to Use	11
Explanation	11
Important	11
10.1.8 Application-Level Authorization	12
Example	12
Explanation	12
10.1.9 Create User Session	13
Explanation	13
10.1.10 Redirect User to Application Dashboard	13
When to Use Refresh Token	14
Refresh Token API Endpoint	14
Request Parameters	14

Java Implementation.....	14
Explanation.....	15
Important.....	15
10.1.12 Logout Implementation.....	16
Logout Code.....	16
Explanation.....	16
10.1.13 Complete Flow Summary.....	17

This section provides a complete end-to-end implementation guide for integrating a Java-based web application (JSP & Servlet) with the Odisha SSO platform using the OpenID Connect (OIDC) Authorization Code Flow.

This implementation ensures:

- Secure authentication via SSO
- Proper token handling
- User session management
- Authorization validation at application level

10.1.1 Prerequisites

Before starting integration, ensure the following details are available from the SSO onboarding team:

- Client ID
- Client Secret
- Realm URL
- Redirect URI (registered in SSO)
- Post Logout Redirect URI

Example Configuration:

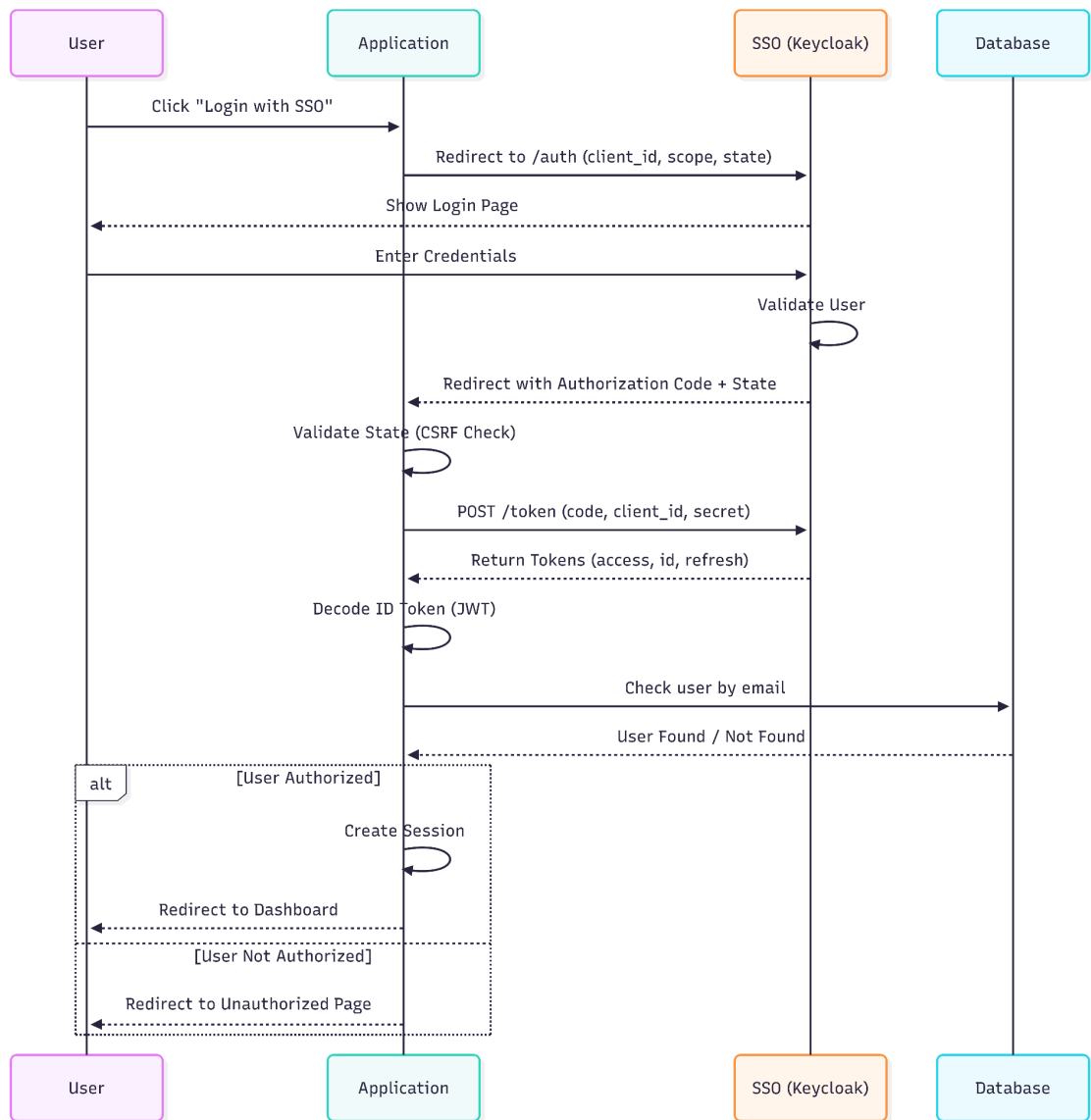
ssoBaseUr1: <https://sso.odisha.gov.in>

realm: stag

client-id: your-client-id

client-secret: your-secret

redirect-uri: <https://yourapp/callback>



10.1.2 Initiate SSO Login (Redirect User to SSO)

When the user clicks "Login with SSO", the application must redirect the user to the SSO authorization endpoint.

JSP Login Button

```
<a href="login">Login using Odisha SSO</a>
```

Servlet Implementation

```
String state = UUID.randomUUID().toString();  
request.getSession().setAttribute("OIDC_STATE", state);
```

```
String authUrl =  
"{ssoBaseUrl}/realms/{realm}/protocol/openid-connect/auth" +  
    "?client_id=" + clientId +  
    "&response_type=code" +  
    "&scope=openid profile email" +  
    "&redirect_uri=" + redirectUri +  
    "&state=" + state;  
response.sendRedirect(authUrl);
```

Explanation

- **state** → Prevents CSRF attacks
- **response_type=code** → Authorization Code Flow
- **scope** → Requests user details (email, profile)
- Redirects user to SSO login page

10.1.3 Receive Authorization Code from SSO

After successful login, SSO redirects the user back:

```
https://yourapp/callback?code=AUTH_CODE&state=xyz
```

Extract Parameters

```
String code = request.getParameter("code");  
String state = request.getParameter("state");  
String sessionState = (String) session.getAttribute("OIDC_STATE");
```

Validate State (Security Check)

```
if (!state.equals(sessionState)) {  
    throw new ServletException("Invalid state parameter");  
}
```

Explanation

- Ensures request is valid
- Prevents replay attacks

10.1.4 Exchange Authorization Code for Tokens

The application exchanges the authorization code for tokens using a POST request.

Token Request Code

```
String tokenUrl =
    "{ssoBaseUrl}/realms/{realm}/protocol/openid-connect/token";

String params = "grant_type=authorization_code" +
    "&code=" + code +
    "&client_id=" + clientId +
    "&client_secret=" + clientSecret +
    "&redirect_uri=" + redirectUri;

URL url = new URL(tokenUrl);
URLConnection conn = (URLConnection) url.openConnection();
conn.setRequestMethod("POST");
conn.setRequestProperty("Content-Type",
    "application/x-www-form-urlencoded");
conn.setDoOutput(true);
OutputStream os = conn.getOutputStream();
os.write(params.getBytes());
os.flush();
```

Explanation

- Exchanges authorization code for tokens
- SSO validates client credentials and code

10.1.5 Receive Token Response

Example Response

```
{
  "access_token": "...",
  "id_token": "...",
  "refresh_token": "...",
  "expires_in": 300
}
```

Read Response

```
BufferedReader reader = new BufferedReader(
    new InputStreamReader(conn.getInputStream()));
StringBuilder responseData = new StringBuilder();
String line;
while ((line = reader.readLine()) != null) {
    responseData.append(line);
}
JSONObject tokens = new JSONObject(responseData.toString());
```

10.1.6 Decode and Validate ID Token

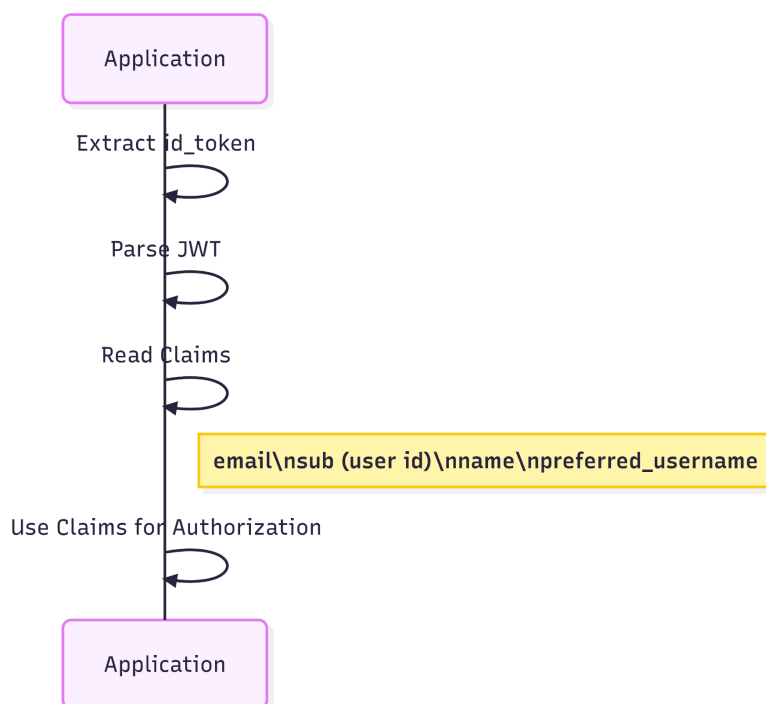
The ID token contains user identity information.

Decode JWT

```
String idToken = tokens.getString("id_token");  
  
SignedJWT jwt = (SignedJWT) JWTParser.parse(idToken);  
  
String email = jwt.getJWTClaimsSet().getStringClaim("email");  
  
String username =  
    jwt.getJWTClaimsSet().getStringClaim("preferred_username");  
  
String userId = jwt.getJWTClaimsSet().getSubject();
```

Explanation

- **email** → user identity
- **sub** → unique user ID in SSO
- **preferred_username** → login ID



10.1.7 Token Validation

Applications must validate the ID Token to ensure it is authentic, untampered, and issued by the trusted SSO server.

Applications should validate:

- Token expiry (**exp**)
- Issuer (**iss**)
- Audience (**aud**)
- Signature using public key

Public Key Endpoint

```
{ssoBaseUrl}/realms/{realm}/protocol/openid-connect/certs
```

Maven Dependency

```
<dependency>  
  <groupId>com.nimbusds</groupId>  
  <artifactId>nimbus-jose-jwt</artifactId>  
  <version>9.31</version>  
</dependency>
```

Token Validation Code Example

```
// Parse ID Token  
SignedJWT signedJWT = (SignedJWT) JWTParser.parse(idToken);  
  
// Fetch public keys from SSO  
URL jwkURL = new URL("{ssoBaseUrl}/realms/{realm}/protocol/openid-connect/certs  
");  
JWKSet jwkSet = JWKSet.load(jwkURL);
```

```
// Get Key ID from token header
String keyID = signedJWT.getHeader().getKeyID();

// Find matching public key
JWK jwk = jwkSet.getKeyByKeyId(keyID);
RSAKey rsaKey = (RSAKey) jwk;

// Create verifier
JWSVerifier verifier = new RSASSAVerifier(rsaKey.toRSAPublicKey());

// Verify signature
if (!signedJWT.verify(verifier)) {
    throw new SecurityException("Invalid token signature");
}

// Validate claims
JWTClaimsSet claims = signedJWT.getJWTClaimsSet();

// Check token expiry
Date expiry = claims.getExpirationTime();
if (expiry == null || new Date().after(expiry)) {
    throw new SecurityException("Token expired");
}

// Validate issuer
String issuer = claims.getIssuer();
if (!issuer.equals("{ssoBaseUrl}/realms/{realm}")) {
    throw new SecurityException("Invalid issuer");
}

// Validate audience
if (!claims.getAudience().contains(clientId)) {
    throw new SecurityException("Invalid audience");
}
```

How to Use

```
String idToken = tokens.getString("id_token");  
// Validate token before using it  
// (Call this before extracting email or user info)
```

Explanation

- Verifies token signature using SSO public key
- Ensures token is not expired
- Confirms token is issued by Odisha SSO
- Validates that token belongs to this application

Important

- Token validation must always be done in backend
- Do not trust token data without verification
- Reject invalid or expired tokens immediately

10.1.8 Application-Level Authorization

SSO authenticates user identity, but application must verify access.

Example

```
User user = userDao.getUserByEmail(email);  
if (user == null) {  
    response.sendRedirect("unauthorized.jsp");  
    return;  
}
```

Explanation

- Only registered users are allowed
- Prevents unauthorized access

10.1.9 Create User Session

```
HttpSession session = request.getSession();  
session.setAttribute("user", user);  
session.setAttribute("access_token", tokens.getString("access_token"));  
session.setAttribute("id_token", idToken);  
session.setAttribute("refresh_token", tokens.getString("refresh_token"));
```

Explanation

- Stores user login state
- Tokens can be reused for APIs

10.1.10 Redirect User to Application Dashboard

After successful authentication, token validation, and session creation, the user is redirected to the application dashboard.

Code

```
response.sendRedirect("dashboard.jsp");
```

Explanation

- Redirects authenticated user to home/dashboard page
- Ensures only logged-in users access protected resources

10.1.11 Refresh Token Flow (Maintaining User Session)

Access tokens have a limited lifespan (e.g., 5 minutes). To avoid forcing the user to log in again after expiry, the application uses the refresh token to obtain a new access token.

When to Use Refresh Token

The application should refresh the access token when:

- Access token has expired (**expires_in** reached)
- API calls return 401 Unauthorized
- Before making a critical backend API call (proactive refresh)

Refresh Token API Endpoint

```
{ssoBaseUrl}/realms/{realm}/protocol/openid-connect/token
```

Request Parameters

```
grant_type=refresh_token
refresh_token=<refresh_token>
client_id=<client_id>
client_secret=<client_secret>
```

Java Implementation

```
String refreshToken = (String) session.getAttribute("refresh_token");

String tokenUrl =
"{ssoBaseUrl}/realms/{realm}/protocol/openid-connect/token";

String params = "grant_type=refresh_token"
    + "&refresh_token=" + refreshToken
    + "&client_id=" + clientId
    + "&client_secret=" + clientSecret;

URL url = new URL(tokenUrl);
```

```

URLConnection conn = (URLConnection) url.openConnection();

conn.setRequestMethod("POST");

conn.setRequestProperty("Content-Type",
"application/x-www-form-urlencoded");

conn.setDoOutput(true);

OutputStream os = conn.getOutputStream();

os.write(params.getBytes());

os.flush();

BufferedReader reader = new BufferedReader(
    new InputStreamReader(conn.getInputStream()));

StringBuilder responseData = new StringBuilder();

String line;

while ((line = reader.readLine()) != null) {
    responseData.append(line);
}

JSONObject tokenResponse = new JSONObject(responseData.toString());

// Update tokens in session

session.setAttribute("access_token",
tokenResponse.getString("access_token"));

session.setAttribute("id_token", tokenResponse.getString("id_token"));

session.setAttribute("refresh_token",
tokenResponse.getString("refresh_token"));

```

Explanation

- Uses stored **refresh_token** to obtain a new access token
- Prevents user from logging in again
- Updates session with new tokens
- Maintains seamless user experience

Important

- Refresh token also has expiry → after expiry, user must re-login
- Always store refresh token securely

10.1.12 Logout Implementation

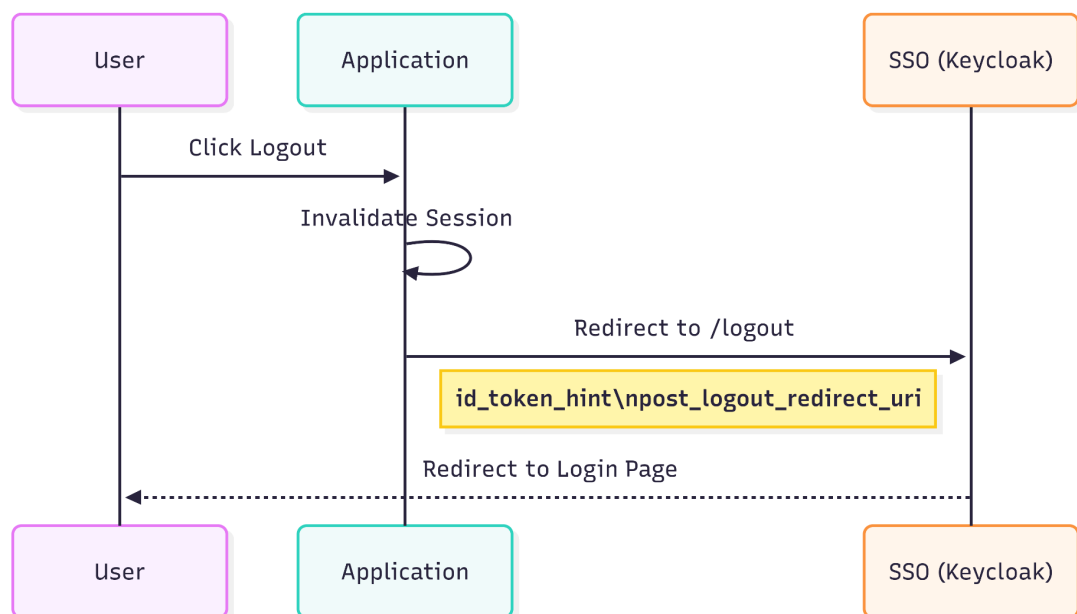
To log out the user from both application and SSO:

Logout Code

```
String logoutUrl =  
"{ssoBaseUrl}/realms/{realm}/protocol/openid-connect/logout" +  
    "?id_token_hint=" + idToken +  
    "&post_logout_redirect_uri=" + redirectUri;  
  
session.invalidate();  
  
response.sendRedirect(logoutUrl);
```

Explanation

- Ends SSO session
- Clears application session



10.1.13 Complete Flow Summary

1. User clicks Login with SSO
2. Application redirects user to SSO Authorization Endpoint
3. User authenticates using Government email
4. SSO redirects back with authorization code and state
5. Application validates state parameter (CSRF protection)
6. Application exchanges authorization code for tokens
7. Application validates ID token (signature, expiry, issuer, audience)
8. Application extracts user details from ID token
9. Application validates user in database (authorization)
10. Session is created for authenticated user
11. User is redirected to application dashboard
12. Access token is refreshed when expired using refresh token
13. Logout is handled via SSO and application session is cleared